

WEEK 2 – ENTITY AND RELATIONSHIP ANALYSIS REPORT

E-Commerce Order Management Database System

Document Type: Analysis & Design Report

Author: Database Architect Team

Status: Approved / Baseline

Target Folder: /Week2/

1. Executive Summary & Overview

Following the successful compilation and verification of the Software Requirement Specification (SRS) in Week 1, this **Entity and Relationship Analysis Report** formalizes the logical data requirements for the *E-Commerce Order Management Database System*. This phase bridges the gap between raw business rules and physical database implementation.

The fundamental objective is to model high-integrity data objects capable of tracking customers, management of product inventories, complex hierarchical ordering components, dynamic payment statuses, and structured shipping logs. This report delivers comprehensive structural lists, explicit primary/foreign key mappings, precise attribute constraints, and cardinality rules to ensure transactional stability and long-term analytics scalability.

2. Entity Identification Report

Based on the operational flow of a modern e-commerce lifecycle, eight primary entities have been isolated. These entities encompass both master data structures (e.g., Customers, Products) and transactional data structures (e.g., Orders, OrderItems, Payments).

Entity Name	Classification	Description and Business Function
Customers	Master Data	Stores unique profile data for registered buyers including authentication traces, contact details, and account lifecycles.
Addresses	Master Data	Manages geographical profiles linked to customers. Supports separate distinct mappings for shipping and billing contexts.
Categories	Master Data	Defines structural grouping for items, enabling nested classifications and catalog optimization queries.
Products	Master Data	Maintains the official store catalog including real-time stock balances, dynamic pricing benchmarks, and category descriptions.
Orders	Transactional	Acts as the core ledger entry documenting a customer purchase intent, total value, state-machine milestones, and point-in-time references.
OrderItems	Transactional	Resolves the many-to-many matrix between Orders and Products, capturing precise historical snapshots of item costs and quantities.

Entity Name	Classification	Description and Business Function
Payments	Transactional	Captures external financial transaction mappings, settlement verifications, payment gateways used, and compliance details.
ShippingDetails	Transactional	Tracks fulfilment operations, logistics carrier integrations, tracking markers, and delivery state tracking.

3. Entity Attribute List & Constraint Matrix

To guarantee strict structural validation at the database engine level, specific constraints—including **Primary Keys (PK)**, **Foreign Keys (FK)**, **Not Null (NN)**, **Unique (UQ)**, **Check Constraints (CK)**, and **Default Values (DF)**—are assigned across all attributes.

3.1. Customers Entity

Attribute	Data Type	Key/Constraint	Default / Check Rules
CustomerID	INT AUTO_INCREMENT	PK	Non-nullable, Auto-generating
FirstName	VARCHAR(50)	NOT NULL	-
LastName	VARCHAR(50)	NOT NULL	-
Email	VARCHAR(100)	NOT NULL , UNIQUE	Must fulfill email regex syntax
Phone	VARCHAR(20)	Optional	-
CreatedAt	TIMESTAMP	NOT NULL	DF = CURRENT_TIMESTAMP
Status	VARCHAR(15)	NOT NULL	DF = 'Active' CK: In ('Active','Suspended')

3.2. Addresses Entity

Attribute	Data Type	Key/Constraint	Default / Check Rules
AddressID	INT AUTO_INCREMENT	PK	Non-nullable
CustomerID	INT	FK (CUSTOMERS) , NN	ON DELETE CASCADE Cascading
AddressLine1	VARCHAR(150)	NOT NULL	-
AddressLine2	VARCHAR(150)	Optional	-
City	VARCHAR(50)	NOT NULL	-

Attribute	Data Type	Key/Constraint	Default / Check Rules
State	VARCHAR(50)	NOT NULL	-
PostalCode	VARCHAR(20)	NOT NULL	-
Country	VARCHAR(60)	NOT NULL	<i>DF = 'United States'</i>
AddressType	VARCHAR(15)	NOT NULL	<i>DF = 'Shipping'</i> CK: In ('Shipping','Billing','Both')

3.3. Categories Entity

Attribute	Data Type	Key/Constraint	Default / Check Rules
CategoryID	INT AUTO_INCREMENT	PK	Non-nullable
Name	VARCHAR(50)	NOT NULL , UNIQUE	-
Description	TEXT	Optional	-

3.4. Products Entity

Attribute	Data Type	Key/Constraint	Default / Check Rules
ProductID	INT AUTO_INCREMENT	PK	Non-nullable
Name	VARCHAR(150)	NOT NULL	Indexed for catalog lookups
Description	TEXT	Optional	-
Price	DECIMAL(10,2)	NOT NULL	CK: Price > 0.00
StockQuantity	INT	NOT NULL	DF = 0 CK: StockQuantity >= 0
CategoryID	INT	FK (CATEGORIES) , NN	ON DELETE RESTRICT Safe-block
Status	VARCHAR(15)	NOT NULL	DF = 'Active' CK: In ('Active','Draft','Archived')

3.5. Orders Entity

Attribute	Data Type	Key/Constraint	Default / Check Rules
OrderID	INT AUTO_INCREMENT	PK	Non-nullable
CustomerID	INT	FK (CUSTOMERS) , NN	ON DELETE RESTRICT Preservation
OrderDate	TIMESTAMP	NOT NULL	DF = CURRENT_TIMESTAMP
TotalAmount	DECIMAL(12,2)	NOT NULL	DF = 0.00 CK: TotalAmount >= 0.00
Status	VARCHAR(20)	NOT NULL	DF = 'Pending' CK: In ('Pending','Paid','Processing','Shipped','Delivered','Cancelled')
ShippingAddressID	INT	FK (ADDRESSES) , NN	Destination verification link
BillingAddressID	INT	FK (ADDRESSES) , NN	Financial trace accounting link

3.6. OrderItems Entity

Attribute	Data Type	Key/Constraint	Default / Check Rules
OrderItemID	INT AUTO_INCREMENT	PK	Non-nullable
OrderID	INT	FK (ORDERS) , NN	ON DELETE CASCADE Cascade sweep
ProductID	INT	FK (PRODUCTS) , NN	ON DELETE RESTRICT Lock prevention
Quantity	INT	NOT NULL	CK: <i>Quantity > 0</i>
UnitPrice	DECIMAL(10,2)	NOT NULL	Historical point-in-time value
Discount	DECIMAL(5,2)	NOT NULL	DF = 0.00 CK: Discount >= 0.00 AND Discount <= 100.00

3.7. Payments Entity

Attribute	Data Type	Key/Constraint	Default / Check Rules
PaymentID	INT AUTO_INCREMENT	PK	Non-nullable
OrderID	INT	FK (ORDERS) , NN , UQ	One payment ledger per order split
PaymentDate	TIMESTAMP	NOT NULL	DF = <i>CURRENT_TIMESTAMP</i>
Amount	DECIMAL(12,2)	NOT NULL	CK: <i>Amount > 0.00</i>
PaymentMethod	VARCHAR(30)	NOT NULL	CK: In ('Credit_Card','PayPal','Stripe','Crypto','Bank_Transfer')
PaymentStatus	VARCHAR(20)	NOT NULL	DF = <i>'Pending'</i> CK: In ('Pending','Completed','Failed','Refunded')
TransactionReference	VARCHAR(100)	UNIQUE	External gateway reference verification

3.8. ShippingDetails Entity

Attribute	Data Type	Key/Constraint	Default / Check Rules
ShippingID	INT AUTO_INCREMENT	PK	Non-nullable

Attribute	Data Type	Key/ Constraint	Default / Check Rules
OrderID	INT	FK (ORDERS) , NN , UQ	One master tracking manifest per order
Carrier	VARCHAR(50)	NOT NULL	e.g., 'FedEx', 'UPS', 'DHL'
TrackingNumber	VARCHAR(100)	UNIQUE	Null permitted until dispatch initialization
ShippingStatus	VARCHAR(20)	NOT NULL	DF = 'Manifest_Created' CK: In ('Manifest_Created', 'In_Transit', 'Out_For_Delivery', 'Delivered', 'Returned')
ShippedDate	TIMESTAMP	Optional	Populated dynamically at exit node scan
EstimatedDelivery	DATE	Optional	Target operational timeline profile

4. Relationship & Cardinality Analysis

Database relational integrity is maintained via relational mapping paradigms. Below is the operational evaluation of the system cross-connections:

1. Customers to Addresses (1 : N)

- **Business Logic:** A customer may establish several delivery nodes (e.g., corporate headquarters, holiday addresses, residential suites) to accommodate flexible shipping scenarios. However, each unique address trace profile strictly links back to a single designated buyer.
- **Cardinality:** One-to-Many. Zero or more child rows in the Address table map back to a required single parent row in the Customer table.

2. Customers to Orders (1 : N)

- **Business Logic:** An active customer returns repeatedly over time to generate subsequent distinct purchase orders. Conversely, an individual order entry belongs definitively to a singular registered entity.
- **Cardinality:** One-to-Many. An order row cannot exist anonymously without an explicit Customer relationship reference.

3. Categories to Products (1 : N)

- **Business Logic:** Inventory items are aggregated into systematic catalog sets (e.g., "Electronics") to handle filter discovery. A category hosts multiple individual products, while each product standardizes classification by pointing back to one primary category tree.
- **Cardinality:** One-to-Many. Restrictive data protection rules ensure parent classifications cannot be expunged while containing items.

4. Orders to OrderItems (1 : N)

- **Business Logic:** A shopper bundles multiple distinct goods inside a single purchase ticket. Each product selection corresponds to a line-item detailing item variations, quantities, and transactional values.
- **Cardinality:** One-to-Many. Eliminating an order prompts an instant cascading purge to clean stale dependent line-items.

5. Products to OrderItems (1 : N)

- **Business Logic:** A popular catalog item is chosen across separate baskets by different shoppers. Each discrete choice maps directly as an item line entry inside those independent transactions. This breaks down the structural Many-to-Many relationship between Orders and Products.
- **Cardinality:** One-to-Many. A core product cannot be dropped from the records if past operational history includes its ID in past line-items.

6. Orders to Payments (1 : 1)

- **Business Logic:** To enforce direct balance accounting and prevent reconciliation drift, every individual checkout transaction corresponds exactly to a single financial transaction payload within the payments ledger.
- **Cardinality:** One-to-One. Enforced by treating the Order ID attribute as a strict Unique Key inside the Payments schema.

7. Orders to ShippingDetails (1 : 1)

- **Business Logic:** For tracking consistency, a single consolidated order standardizes tracking under one unified logistical master air bill routing record.
- **Cardinality:** One-to-One. Secured by placing a unique constraint over the Order ID field inside the Shipping data table.

5. Logical ER Schema Mapping Architecture

The structural relationship layout can be visualized logically below. This table maps how foreign key fields establish join hierarchies across the operational environment:

Source Table	Foreign Key Field	References Parent Table	Enforcement Rule / Action
Addresses	CustomerID	Customers (CustomerID)	ON DELETE CASCADE (Purge logs)
Products	CategoryID	Categories (CategoryID)	ON DELETE RESTRICT (Lock category)
Orders	CustomerID	Customers (CustomerID)	ON DELETE RESTRICT (Block if order exists)
Orders	ShippingAddressID	Addresses (AddressID)	ON DELETE RESTRICT (Keep route details)

Source Table	Foreign Key Field	References Parent Table	Enforcement Rule / Action
Orders	BillingAddressID	Addresses (AddressID)	ON DELETE RESTRICT (Keep accounting details)
OrderItems	OrderID	Orders (OrderID)	ON DELETE CASCADE (Purge items with invoice)
OrderItems	ProductID	Products (ProductID)	ON DELETE RESTRICT (Keep transaction record)
Payments	OrderID	Orders (OrderID)	ON DELETE RESTRICT (Keep payment audits)
ShippingDetails	OrderID	Orders (OrderID)	ON DELETE RESTRICT (Keep freight history)

6. Visual Structural Blueprint Mappings

The structural setup below shows the internal schema layouts of the core entities side by side to simplify the database generation step.

CUSTOMERS PK: CustomerID [INT] FirstName [VARCHAR] LastName [VARCHAR] Email [VARCHAR] (UQ) Phone [VARCHAR] CreatedAt [TIMESTAMP] Status [VARCHAR]	ORDERS PK: OrderID [INT] FK: CustomerID → CUSTOMERS FK: ShippingAddressID → ADDRESSES FK: BillingAddressID → ADDRESSES OrderDate [TIMESTAMP] TotalAmount [DECIMAL] Status [VARCHAR]
PRODUCTS PK: ProductID [INT] FK: CategoryID → CATEGORIES Name [VARCHAR] Description [TEXT] Price [DECIMAL] StockQuantity [INT] Status [VARCHAR]	ORDER_ITEMS PK: OrderItemID [INT] FK: OrderID → ORDERS FK: ProductID → PRODUCTS Quantity [INT] UnitPrice [DECIMAL] Discount [DECIMAL]

Prepared By: Lead Data Architect
Systems Engineering Division

Reviewed & Certified By: Quality Assurance Board
E-Commerce Platform Core Team